

Original article

Development of a software architecture for bioprocess modeling

Priscila Marques da Paz^a, Caroline Satye Martins Nakama^b, Galo Antonio Carrillo Le Roux^a,*^a Department of Chemical Engineering, Polytechnic School, University of São Paulo, Av. Prof. Luicano Gualberto, trav. 3, 380, São Paulo, 05508-010, São Paulo, Brazil^b Department of Chemical Engineering, Norwegian University of Science and Technology, Høgskoleringen 1, Trondheim, 7034, Trøndelag, Norway

ARTICLE INFO

Keywords:

Bioprocess

Ontology

Modeling

Unified modeling language

ABSTRACT

Increasing the productivity of a biotechnological process becomes feasible through the development of Process Systems Engineering tools, which integrate experimental data with mathematical modeling. This work aims to develop a software architecture for modeling bioprocesses that is accessible to a multidisciplinary group. To achieve this aim, the software must be thoroughly designed based on an ontology that describes bioprocesses that can be apprehended by researchers from different fields. The ontological representation is carried out using Unified Modeling Language diagrams, whose use is demonstrated by a parameter estimation case study. It is concluded that good software development practices can be provided through the proposed architecture, since it guides simulations and parameter estimations of biotechnological processes in a structured way.

1. Introduction

Microorganisms are the heart of biotechnological processes, as they are the agents responsible for synthesizing complex molecules that can be difficult to obtain by chemical means, in addition to promoting sustainability. However, to make production economically viable, it is necessary to design the bioprocess and use Process Systems Engineering (PSE) tools (Doran, 2013; Hemmerich et al., 2021). Furthermore, experimental data acquisition techniques have evolved over the years, resulting in a steadily increase in the experimental dataset sizes, that often only quantitative methods are capable of processing and interpreting them. Therefore, combining PSE tools with biotechnological skills becomes fundamental, with the need to develop tools that facilitate work and collaboration in multidisciplinary groups. This is essential for bioprocesses modeling in general, which benefits multiple approaches, and among them, the area of systems biology stands out (Meyer and Saez-Rodriguez, 2021).

Although systems biology has become indispensable to deal with the increasing amount of data (which are generated by improving the collection of information from various sources, such as genomics, transcriptomics, and metabolomics), it should be seen as a complementary approach to traditional modeling techniques, which have already demonstrated success in bioprocesses over the years. Classical and empirical mechanistic models, for example, still play an essential role in bioreactor modeling, but they tend to focus on isolated parts or specific aspects of biotechnological processes, while systems biology takes into

account the interaction between multiple components at the same time, that is, it considers entire biological networks (Shuler and Kargi, 2002; Wilkinson, 2006). Therefore, integrating these traditional approaches with the innovations brought by systems biology is the ideal path to more robust and versatile models (Bassalo and Gill, 2016; Meyer and Saez-Rodriguez, 2021).

As new information is collected and techniques developed, it is possible to incorporate new principles studied into models, making them increasingly complete and accurate (Bassalo and Gill, 2016; Meyer and Saez-Rodriguez, 2021). Due to the demand for information, hardly a single person will retain a deep knowledge of a given area. Thus, systems biology, as part of an arsenal of approaches, is a multidisciplinary topic and demands more than one professional involved to integrate technological and scientific advances (Meyer and Saez-Rodriguez, 2021).

In this scenario, it is desirable that computational structures are developed to guide bioprocess modeling applications. As researchers with different backgrounds will be able to use and contribute to software updates, it is recommended that the structure is well designed. It is appropriate for the entire system to be structured before coding, as the size, complexity and scope of a system can increase, even in simple cases (Guedes, 2018). To ensure the (re)usability of the system, a way to standardize models applied to biomanufacturing processes is sought. Typically, diagrams are used so that everyone agrees on the symbols used and how they are organized. Thus, establishing standardized

* Corresponding author.

E-mail address: galaroux@usp.br (G.A.C. Le Roux).URL: <http://lattes.cnpq.br/9084088181088883> (G.A.C. Le Roux).

graphical notions becomes important for the transmission of biological knowledge between different communities (Novère et al., 2009).

According to Porubsky et al. (2020), some of the most widely adopted modeling standards in systems biology are SBML, CellML, BioNetGen, SED-ML, BioPAX, and SBGN. The first three are characterized by being modeling language standards, which represent and encode biological models. SBML (Systems Biology Markup Language) is used to communicate and store computational models of biological processes and can represent metabolic and regulatory networks, metabolism, and cellular signaling pathways (Hucka, 2013). There are several SBML models available in public repositories and also converters to SBML. Compared to SBML, CellML has a generic structure with broader application, and is used to specify mathematical models of biological systems, with a focus on supporting problems formed by differential algebraic equations (DAEs) (Kohl, 2011; Miller et al., 2013). The mathematical expressions of a CellML model are embedded in components, which represent concrete or abstract concepts (Miller et al., 2013). BioNetGen language (BNGL) is used to code the open-source software package of the same name, BioNetGen, which is used to build and simulate rule-based models. This approach incorporates combinatorial complexity into models of biochemical systems. Instead of manually enumerating all possible species and reactions within a system, the rule-based model defines only the reactive motifs within macromolecular complexes and the interactions and modifications that involve them. Specific user-defined actions can be added to a BNGL model file, and models can be exported to different formats, such as SBML (Harris et al., 2016).

The modeling standard SED-ML (Simulation Experiment Description Markup Language) is a simulation description language. Descriptions about the simulation are encoded, including simulator settings and information about the models, as well as changes to them. It has an XML format and is independent of software tools used to run the simulations. BioPAX is another proposed XML-based standard that can interact with available metabolic pathway databases as well as other languages such as SBML and CellML. BioPAX is a modeling language that represents metabolic and signaling pathways, molecular interactions, and regulatory networks (Gu and Sauro, 2014; Porubsky et al., 2020). Another standardization alternative is the graphical format for representing biological networks and cellular processes, which is the case of SBGN (Systems Biology Graphical Notation). SBGN allows a complex biological model to be visualized graphically, helping independent groups to understand its components and interactions (Porubsky et al., 2020). Information is represented through graphical objects, or glyphs, and is organized in a structure. It is based on descriptions of processes, entity relationships and activity flows. It has visually complemented SBML, already presented (Schreiber and Le Novère, 2013; Gu and Sauro, 2014).

As the modeling field has grown, it has become clear there is a need to supplement data and models with semantic content. This additional semantic information also needs to be in a standard format, so it can be easily integrated and interpreted by other systems and third parties. Thus, a common language is needed so that different models can describe the same characteristics of biotechnological processes. This approach is achieved by annotating data using common and structured vocabularies, which gives rise to the concept of ontology (Juty and le Novère, 2013; Gu and Sauro, 2014).

Ontologies are a support option for organizing information (Guedes, 2018), a formal system for representing knowledge (Demir et al., 2010). Ontology is a specification of a concept, and usually involves classes, their relationship, and axioms to describe the intended semantics. It is used in computer science and has become widespread in areas such as medicine, biology and chemistry. OntoCAPE ontology for Chemical Engineering (Marquardt, 2010) should be highlighted. However, there is still a lack of works in the bioprocess area. The work of Duong-Trung et al. (2023) suggests that an appropriate ontology can boost the area of bioprocesses in machine learning, which favors automation

and data modeling. Nowadays, there are free tools that help in bioprocess data analysis, such as AnaBioPlus (Oliveira et al., 2017) and pyFOOMB (Hemmerich et al., 2021), but without providing a complete ontology of their development, which can prevent contributions and improvements are adequately carried out.

Information becomes readily shared through the ontology, which in this work provides the instantiation of bioprocess components, standardizing and organizing them. In the literature, modeling standards such as SBML, SED-ML and BioPAX are complemented by ontology through some tools such as Systems Biology Ontology (SBO), Web Ontology Language (OWL) and Unified Modeling Language (UML) (Roux-Rouquié et al., 2004; Demir et al., 2010; Hoehndorf et al., 2011; Juty and le Novère, 2013). The tool that most helps for the purpose of this work is UML, as it uses combinations of several models to describe its multifaceted aspects. It is widely recognized in the software industry and avoids the implementation of inconsistencies (Arora et al., 2020; Yurin and Dorodnykh, 2020). It can become complex in detailed projects, but it is a standardized visual language to facilitate communication (Arcuri, 2018).

Thus, the objective of this work is to develop a software architecture for bioprocess modeling, so this structure can be used and reused in different scenarios and configurations in the field, without compromising the code or malfunctioning. For this purpose, an ontology of bioprocess components was developed with UML support, and the open-source language Julia was chosen to implement methods and techniques that describe bioprocesses. Case studies are presented and discussed to exemplify the use of UML for obtaining a generalized software.

2. Methodology

2.1. Structure and development of the ontology

From the philosophical interest in the conceptualization of the world, ontology arises. It is based on classifying things, mainly based on the observation of common characteristics that are shared between a collection of components. It can be organized in a hierarchical way, where each inherited property is manifested in subclassifications. The components of the same created class are called instances and the characteristics of the class are attributes. Each attribute can have one or more values specific to the class to which it belongs (Marquardt, 2010; Zhang et al., 2013).

The UML diagrams, which represent the ontology, were implemented using the StarUML software (version 5.0.2). Two types of diagrams were chosen: use case and class diagram. The first presents an external view of the functionality the system should offer to users, without detailing how this will be implemented. It is composed by actors and use cases. The actors can be users, other systems or some external hardware that will use the system in some way, and in all cases they are represented by “stick figures”, which are drawings of a person, where the head is a circle and the body is composed of several straight lines. A “stick figure” makes up Fig. 2 and is called by “User”, since in this work the actor that will enter data will be the user himself. Use cases are the functionalities that the system provides to actors, and the diagram has an ellipse format containing a succinct text that describes action, starting with a verb. Interactions between them are represented by dashed lines, with optional (“extend”) or mandatory (continuous and “include”) actions (Guedes, 2018).

The second chosen diagram includes classes, interfaces and the way they relate. Each class is a set of objects, providing a set of attributes and operations that an instance can perform. Actions happen through operations, which are procedures. More than one interface can be implemented by a class, and different classes can implement each interface. Interactions are represented in a similar way to the use case diagram, but with more detailed options (Guedes, 2018).

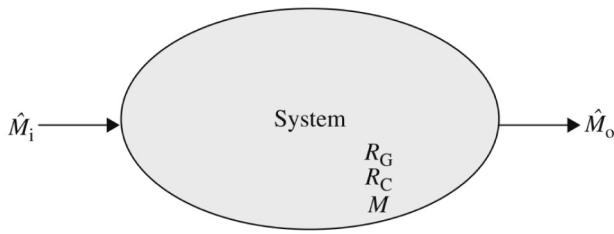


Fig. 1. Flow system for an unsteady-state mass balance.
Source: Doran (2013).

2.2. Bioprocess modeling

Abstraction and classification of bioprocess components are based on equations that are as generic as possible, which allows additions of more complex concepts related to the component, either in the class itself or through interactions with other created ones. Main classes were created for the operating mode of a bioreactor (*OperationMode*), cell growth (*CellGrowth*) and cell death (*CellDeath*), aeration (*Aeration*), yield (*Yield*), discretized ordinary differential equations (*ode*) and estimators for the case of parameter estimation (*estimators*). For each case, child classes are also created and inherit characteristics from the respective main class. For example, *OperationMode* class abstraction is based on the overall mass balance, given by Eq. (1) (Doran, 2013):

$$\frac{dM}{dt} = \hat{M}_i - \hat{M}_o + R_G - R_C \quad (1)$$

where M is the mass of a component in the system, the derivative dM/dt is the accumulated mass within the system and it is zero if the system is in steady-state. $\hat{M}_i$ is the mass entering through system boundaries, i.e., the mass flow rate of a component entering the reactor, $\hat{M}_o$ is the mass flow rate leaving the system, R_G is the mass generation rate of a component, and R_C is the mass consumption rate. All of this is summarized by Fig. 1.

The heir classes are created for components such as biomass, substrate and product, varying in: continuous mode (named as *continuous* for biomass), which considers the entire equation and may have recycle (*continuouswr*); batch mode (*batch* class), which has $\hat{M}_i$ and $\hat{M}_o$ equal to zero; and fedbatch mode (*fedbatch* class), where $\hat{M}_o$ is zero.

When a bioprocess is carried out by microorganisms, the R_G is given by:

$$R_G = \mu_X X \quad (2)$$

where μ_X is the specific growth rate. A similar situation can be found for R_C with a specific death constant.

It can be seen that μ_X from R_G interacts with the *CellGrowth* class and, since μ_X can occur in different ways, for example, with or without substrate or product inhibition, *CellGrowth* will be composed of subclasses that represent all these alternatives to obtain μ_X . If the substrate S is limiting and concentrations changes of other components have no effect on μ_X , equations such as Monod, Tessier, Contois or others can describe this behavior. On the other hand, when substrate or product concentrations are high and inhibitory substances are present in the medium, μ_X depends on the inhibitor concentration. Therefore, these situations can be described by other equations (Shuler and Kargi, 2002; Alterthum et al., 2020).

Cell maintenance and yield coefficients are some of the stoichiometrically related parameters for growth kinetics, which are also influenced by environmental conditions. A set of factors must be considered for classes to emerge and interact with each other, in addition to new classes being created. For example: in a bioprocess carried out aerobically, oxygen becomes a limiting factor for growth and therefore the *Aeration* class must be considered, with subclasses describing the oxygen transfer rate (*OTR*) from the gas phase to the liquid phase, and the oxygen consumption rate (*OUR*) (Shuler and Kargi, 2002). In short, different situations and paths for assembling models are offered by the architecture.

2.3. Case studies

2.3.1. Case study 1: Penicillin fermentation

UML diagrams that describe the system were used to guide the software programming. The language chosen to implement the code is Julia, mainly due to its high performance and dynamic flexibility, in addition to being open source.

To evaluate the code functionality, the parameters estimation of a bioprocess is performed. The case study is described by Mostoufi and Constantinides (2023) and focuses on the production of penicillin. Cultivation is carried out by the microorganism *Penicillium chrysogenum* in a batch fermenter under carefully controlled conditions. The cell growth rate can be modeled by the logistic law (Eq. (3)), and is often used to describe the dynamics of cell growth. This option for cell growth has already been implemented as one of the options to obtain μ_X and in this case, the parameters are called k_1 and k_2 , which are equivalent to μ_X and X_M respectively, and X_M is the maximum amount of biomass achieved by the medium. The term $(1 - X/k_2)$ represents the limiting factor, which reduces growth as the biomass X approaches the value of X_{max} . The biomass concentration is represented by X and R_G is similar to Eq. (2), with the addition of the limiting factor term. As there is no mention of cell death, it is assumed that R_C is equal to zero.

$$\frac{dX}{dt} = k_1 X \left(1 - \frac{X}{k_2} \right) \quad (3)$$

For penicillin production, the expression is similar to Eq. (4), also previously considered in the software and generalized by Eq. (5), which describes the mass balance of product P and is based on Eq. (1). However, q_P and k_{DP} were represented by k_3 and k_4 in the example.

$$\frac{dP}{dt} = q_P X - k_{DP} P \quad (4)$$

$$\frac{dP}{dt} = R_{G,P} - R_{C,P} \quad (5)$$

Two runs of experiments were provided by Mostoufi and Constantinides (2023), containing both cell concentration data X , in percent dry weight, and penicillin concentration data P , in units/mL. The average of X and P experimental data was used, and no normalization was performed. The chosen solver was IPOPT and the objective function is to minimize the sum of squared errors between the experimental data and the model output. The initial conditions are $X = 0.29\% \text{DW}$ and $P = 0 \text{ u/mL}$. The initial estimates for parameters k_1 , k_2 , k_3 and k_4 were 0 h^{-1} .

2.3.2. Case study 2: Lipase production by *pichia pastoris*

In this case study by Surribas et al. (2006), online fluorescence sensors are used to monitor state variables in induced and non-induced cultivations of *Pichia pastoris* for the production of *Rhizopus oryzae* lipase (ROL).

According to Surribas et al. (2006), the yeast *Pichia pastoris* is used in academic and industrial biotechnology for the production of a wide variety of heterologous proteins, and combines the ability to grow in minimal medium with high cell density. Proteins are produced mainly under the control of the alcohol oxidase 1 (AOX1) promoter and through two phases: non-induced and induced. The first phase is associated with batch growth to generate biomass using glycerol as a carbon source, which has a high biomass yield. In the second phase, methanol is added to the culture and it is in this phase that the production of the ROL protein occurs. Therefore, monitoring and controlling biomass, substrate (glycerol and methanol) and heterologous protein are the main variables to obtain high cell density and product yields.

Experimental data on biomass and both substrates are available in the cited work. It is worth noting that the focus of Surribas et al. (2006) was not to estimate the kinetic parameters of the bioprocess, but rather to evaluate the performance of the phases described for lipase production, mainly through fluorescence data collected (not available

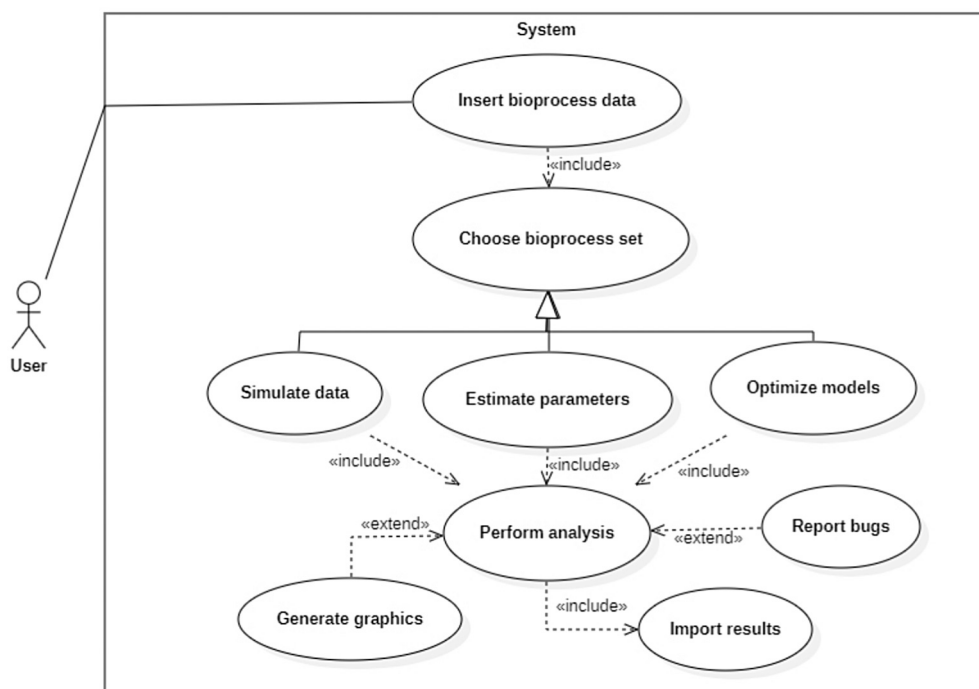


Fig. 2. UML use case diagram of bioprocess system.

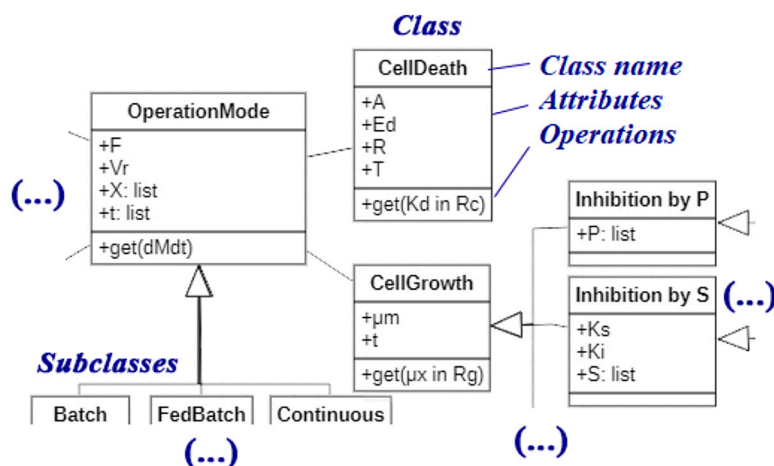


Fig. 3. UML class diagram fragment of bioprocess system.

in the paper). However, the present work presents an estimation of the parameters made in two stages, corresponding to the two mentioned phases: (i) *glycerol fermentation*, which is commonly performed in the cultivation of *P. pastoris*. In this step, a high level of biomass is obtained without induction of recombinant proteins; and (ii) *methanol fermentation phase*, where the production of the ROL protein is induced by the supplied carbon source (Surribas et al., 2006). The parameter estimates for both phases were performed using Eqs. (1) and (2), considering the batch operation mode, and the cell growth described by the Monod equation.

3. Results and discussion

3.1. UML diagrams

The software architecture is shown by the use case diagram (Fig. 2) and the class diagram (Fig. 3).

Fig. 2 shows the cascade of actions, from inserting bioprocess data into the system to generating and importing results. The idea is that the users indicate what they would like to do in the system according to the data they have: simulate the bioprocess, estimate parameters, or even optimize them. In Fig. 3, the previously mentioned classes are represented in a fragment of the total diagram, and the three dots symbolize that there are more subclasses considered. This indicates the class diagram is dynamic, as the software architecture is, and grows depending on what is added to the model.

3.2. Case studies

3.2.1. Case study 1: Estimation of penicillin production parameters

Fig. 4 shows the value of X and P , both experimental and obtained by the model. The final values of the estimated parameters are $k_1 = 3.99 \text{ h}^{-1}$, $k_2 = 4.11\% \text{ DW}$, $k_3 = 1.39 \text{ h}^{-1}$ and $k_4 = 1.61 \text{ h}^{-1}$. A flowchart with an overview of data and functions which are called for parameter estimation is shown in Fig. 5.

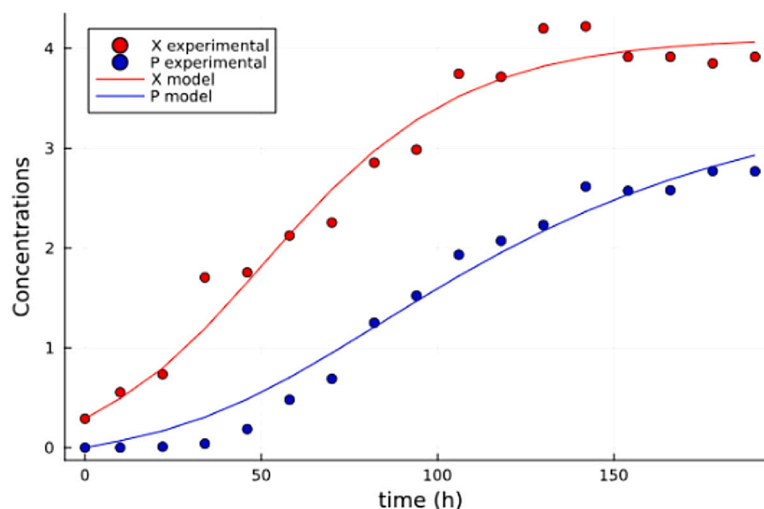


Fig. 4. Case study biomass and penicillin concentrations.

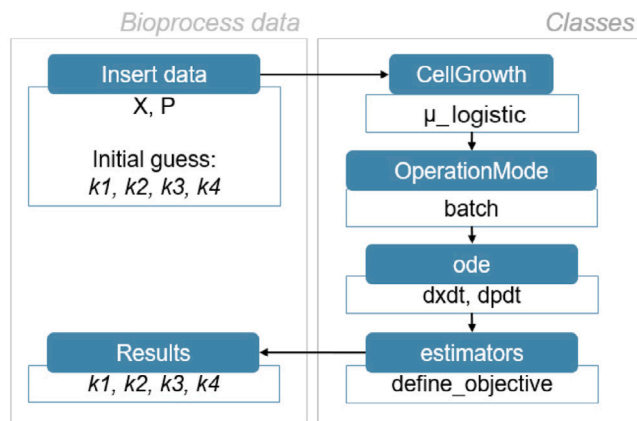


Fig. 5. Kinetic parameter estimation flowchart.

For this work, the case study aims to validate the functioning of the software, mainly through the (re)use of equations, instead of validating the biological model itself. Furthermore, in this specific example, the authors' focus of discussion was curve fitting, not the expected biological meaning of the parameters. However, it is noted the results provided are consistent with the expected behavior and observed in the graph. Penicillin production follows the growth of the microorganism's biomass, with the concentration of P increasing during the growth phase of X , resulting in the value of k_3 , or q_P , which represents the progressive increase in penicillin in the graph. k_4 , on the other hand, is related to penicillin degradation, which stabilizes the product curve after about 150 h and does not continue to rise indefinitely. The value of k_1 indicates rapid growth and k_2 the maximum reached by the biomass, which stabilizes after 120 h.

Fig. 5 is a flowchart that shows the operations performed by the framework. Its left side refers to the bioprocess data, both input and output. In this example, biomass and product concentration data are entered with initial guesses of the parameters. The right side of the figure shows all the classes called and subclasses used, according to the nomenclature given in the code. The first class to be called is *CellGrowth*, where the logistic equation was implemented as a subclass, and is chosen among the various ways available in the code to obtain growth rate. Then, the *batch* operation mode is chosen within the main class *OperationMode* and, through R_G (Eq. (2)), it is related to the

class *CellGrowth*. Since *OperationMode* is used with ordinary differential equations, the *ode* class is called to indicate which equations are discretized. Finally, the *estimators* class is used, where the objective function *define_objective* is selected and minimizes the sum of squared errors between the experimental data and the model output.

3.2.2. Case study 2: Estimation of lipase production parameters

The biomass and glycerol measured both experimentally (dots) and predicted by the model (solid line) are plotted in Fig. 6. The final estimated parameter values are approximately $\mu_m = 1.80 \text{ h}^{-1}$, $K_s = -9.97e^{-9}$, and $Y_{x/s} = 1.00 \text{ g/g}$. The biomass and methanol are also measured experimentally (dots) and predicted by the model (solid line) and plotted in Fig. 7. The final estimated parameter values are $\mu_m = 1.5 \text{ h}^{-1}$, $K_s = 0.11$, and $Y_{x/s} = 0.54 \text{ g/g}$. A flowchart with an overview of the data and functions that are called for parameter estimation was also considered and is shown in Fig. 8. Since the same classes were executed in both phases, changing only the input data values, this flowchart represents both situations.

Analyzing μ_{max} in both cases, the rate estimated by the model is relatively high and, although it is expected that there will be cell growth at a lower rate in the methanol phase, since there is protein induction, the values presented may indicate the Monod Equation is not the most appropriate for this case. In addition, the value of K_s in the glycerol phase is negative and extremely small, which has no biological meaning and may indicate a numerical error. It can be assumed that K_s in this case is zero, but this is not realistic. For the methanol phase, the value presented is reasonable, indicating a certain affinity of the microorganism for the substrate. The values of $Y_{x/s}$ indicate how much biomass is generated per gram of substrate consumed.

As explained in the previous case, Fig. 8 shows the flowchart of classes used by the system, which are necessary to perform the operations. The biomass and substrate concentration data are entered with initial guesses of the parameters. The *CellGrowth* class is called, where the Monod equation was selected (named as subclass *μ_Slim*), and can be changed to any other equation that best describes the model. Since the original paper does not address this, another model can be studied and changed. Following the operations, the classes *OperationMode*, *ode* and *estimators* are used, with respective subclasses chosen and instantiated.

It is important to highlighting that the programming as it was done, together with the ontology, allows new alternative methodologies to be included and tested, which is the biggest advantage in relation to code generated in an unstructured way. Each class can call a different methodology without compromising the code (as showed in Figs. 5 and 8), as long as it belongs to the same class. In addition, increasing

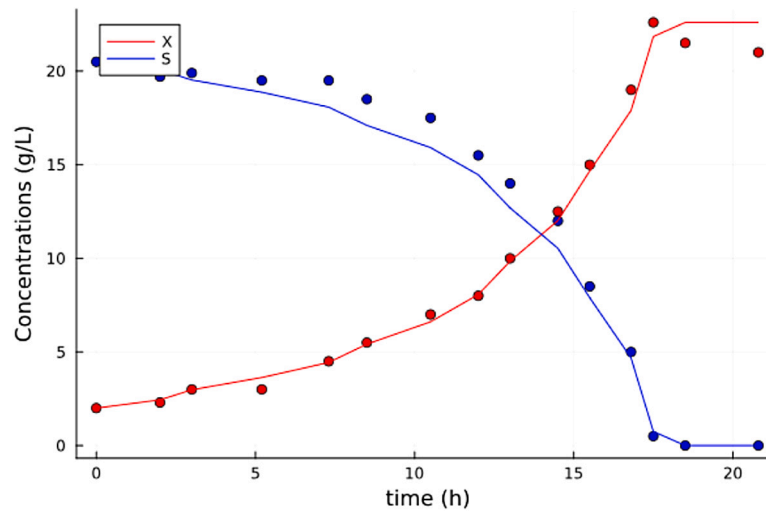


Fig. 6. Biomass and glycerol concentrations in batch culture of *P. pastoris*.

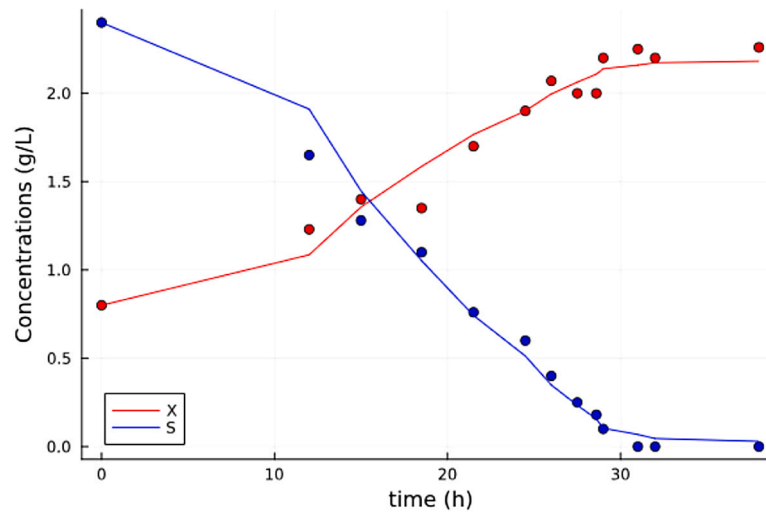


Fig. 7. Biomass and methanol concentrations in batch culture of *P. pastoris*.

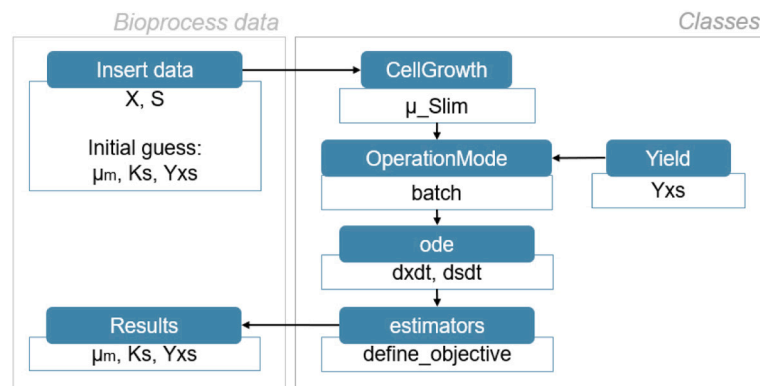


Fig. 8. Kinetic parameter estimation flowchart for lipase production.

the number of equations in the model can make differentiation more laborious. Therefore, the Julia code was adjusted to the JuMP library, allowing greater flexibility and efficiency in the process of building mathematical models. JuMP is a language embedded in Julia, it has domain-specific algebraic modeling with a syntax similar to natural mathematical expressions (Dunning et al., 2017).

JuMP only comprises algebraic equations in its code, and therefore, the differential equations (ODE) were discretized using Implicit Euler. The method belongs to a first-order orthogonal collocation group; however, higher orders of this method can be incorporated to deal with stiff problems (Biegler, 2010). It is interesting to consider this aspect to cover more complex bioprocesses that may involve stiff ODE systems.

4. Conclusion

The developed software architecture can bring new perspectives to bioprocess modeling, as this approach is currently considered a key piece in system design by the software community, especially with the progression of the area of machine learning in bioprocesses. Although the framework is flexible enough to include new models without compromising the code, offering more and more options that result in improvements to data adjustments and estimations, it does not offer an automatic recommendation in the sense of “choosing” the ideal model autonomously for the user. Each bioprocess is unique and depends on many variables, making it difficult to completely automate the choice of the best bioprocess, such as the best kinetics, for example. This is a challenge that can be considered in future work, which would add to the current capacity of the software to be flexible even if the user still needs to choose which path to follow. The case studies in this work exemplify the generalization of software through the use of UML diagrams. From these results, it is clear that the developed architecture can guide good practices for developing simulations and estimates of bioprocess parameters. In summary, this prototype becomes a PSE tool that integrates and analyzes model data, creating a new and robust tool in systems biology.

CRediT authorship contribution statement

Priscila Marques da Paz: Writing – review & editing, Writing – original draft, Methodology, Formal analysis, Conceptualization. **Caroline Satye Martins Nakama:** Methodology, Conceptualization. **Galo Antonio Carrillo Le Roux:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was financially supported by the Coordination of Superior Level Staff Improvement (CAPES), Brazil grant no. 88887.464619/2019-00, PROEX program, National Council for Scientific and Technological Development (CNPq), Brazil for the scholarship for Priscila M. da Paz (142147/2019-2) and RCGI/FAPESP, Brazil (2020/15230-5). Professor Galo A. C. Le Roux acknowledges CNPq, Brazil for his productivity fellowship (311550/2022-3).

References

- Alterthum, F., Schmidell, W., Lima, U.d.A., Moraes, I.d., 2020. *Second ed. Biotecnologia Industrial - Fundamentos*, vol. I. Blücher, São Paulo.
- Arcuri, A., 2018. An experience report on applying software testing academic results in industry: we need usable automated test generation. *Empir. Softw. Eng.* 23, 1959–1981. <http://dx.doi.org/10.1007/s10664-017-9570-9>.
- Arora, V., Singh, M., Bhatia, R., 2020. Orientation-based ant colony algorithm for synthesizing the test scenarios in UML activity diagram. *Inf. Softw. Technol.* 123, 1–21. <http://dx.doi.org/10.1016/j.infsof.2020.106292>.
- Bassalo, M., Gill, R., 2016. Directed evolution and synthetic biology applications to microbial systems. *Curr. Opin. Biotechnol.* 39, 126–133. <http://dx.doi.org/10.1016/j.copbio.2016.03.016>.
- Biegler, L.T., 2010. *Nonlinear Programming*, First ed. Society for Industrial and Applied Mathematics, <http://dx.doi.org/10.1137/1.9780898719383>.
- Demir, E., Cary, M.P., Paley, S., Fukuda, K., Lemer, C., Vastrik, I., Wu, G., D'Eustachio, P., Schaefer, C., Luciano, J., Schacherer, F., Martinez-Flores, I., Hu, Z., Jimenez-Jacinto, V., Joshi-Tope, G., Kandasamy, K., Lopez-Fuentes, A.C., Mi, H., Pichler, E., Rodchenkov, I., Splendiani, A., Tkachev, S., Zucker, J., Gopinath, G., Rajasimha, H., Ramakrishnan, R., Shah, I., Syed, M., Anwar, N., Babur, O., Blinov, M., Brauner, E., Corwin, D., Donaldson, S., Gibbons, F., Goldberg, R., Hornbeck, P., Luna, A., Murray-Rust, P., Neumann, E., Ruebenacker, O., Samwald, M., van Iersel, M., Wimalaratne, S., Allen, K., Braun, B., Whirl-Carrillo, M., Cheung, K.H., Dahlquist, K., Finney, A., Gillespie, M., Glass, E., Gong, L., Haw, R., Honig, M., Hubaut, O., Kane, D., Krupa, S., Kutmon, M., Leonard, J., Marks, D., Merberg, D., Petri, V., Pico, A., Ravenscroft, D., Ren, L., Shah, N., Sunshine, M., Tang, R., Whaley, R., Letovsky, S., Buetow, K.H., Rzhetsky, A., Schachter, V., Sobral, B.S., Dogrusoz, U., McWeeney, S., Aladjem, M., Birney, E., Collado-Vides, J., Goto, S., Hucka, M., Le Novère, N., Maltsev, N., Pandey, A., Thomas, P., Wingender, E., Karp, P.D., Sander, C., Bader, G.D., 2010. The bioPAX community standard for pathway data sharing. *Nat. Biotechnol.* 28 (9), 935–942. <http://dx.doi.org/10.1038/nbt.1666>.
- Doran, P., 2013. *Bioprocess Engineering Principles*, Second ed. Elsevier, Academic Press, Oxford.
- Dunning, I., Huchette, J., Lubin, M., 2017. JuMP: A modeling language for mathematical optimization. *SIAM Rev.* 59 (2), 295–320. <http://dx.doi.org/10.1137/15M1020575>.
- Duong-Trung, N., Born, S., Kim, J.W., Schermeyer, M.-T., Paulick, K., Borisyak, M., Cruz-Bournazou, M.N., Werner, T., Scholz, R., Schmidt-Thieme, L., Neubauer, P., Martinez, E., 2023. When bioprocess engineering meets machine learning: A survey from the perspective of automated bioprocess development. *Biochem. Eng. J.* 190, 108764. <http://dx.doi.org/10.1016/j.bej.2022.108764>.
- Gu, S., Sauro, H., 2014. Chapter 8 - standards, platforms, and applications. In: Kriete, A., Eils, R. (Eds.), *Computational Systems Biology* (Second Edition), Second ed. Academic Press, Oxford, pp. 133–167. <http://dx.doi.org/10.1016/B978-0-12-405926-9.00008-3>, URL <https://www.sciencedirect.com/science/article/pii/B9780124059269000083>.
- Guedes, G., 2018. *UML 2 - Uma abordagem Prática*, Second ed. Novatec, São Paulo.
- Harris, L.A., Hogg, J.S., Tapia, J.-J., Sekar, J.A.P., Gupta, S., Korsunsky, I., Arora, A., Barua, D., Sheehan, R.P., Faeder, J.R., 2016. BioNetGen 2.2: advances in rule-based modeling. *Bioinformatics* 32 (21), 3366–3368. <http://dx.doi.org/10.1093/bioinformatics/btw469>, arXiv:https://academic.oup.com/bioinformatics/article-pdf/32/21/3366/49021661/bioinformatics_32_21_3366.pdf.
- Hemmerich, J., Tenhaef, N., Wiechert, W., Noack, S., 2021. pyFOOMB: Python framework for object oriented modeling of bioprocesses. *Eng. Life Sci.* 21, 242–257. <http://dx.doi.org/10.1002/elsc.202000088>.
- Hoehndorf, R., Dumontier, M., Gennari, J.H., Wimalaratne, S., de Bono, B., Cook, D.L., Koutsos, G.V., 2011. Integrating systems biology models and biomedical ontologies. *BMC Syst. Biol.* 5 (124).
- Hucka, M., 2013. Systems biology markup language (SBML). In: Dubitzky, W., Wolkenhauer, O., Cho, K.-H., Yokota, H. (Eds.), *Encyclopedia of Systems Biology*. Springer, New York, NY, pp. 2057–2063. http://dx.doi.org/10.1007/978-1-4419-9863-7_1091.
- Juty, N., le Novère, N., 2013. Systems biology ontology. In: Dubitzky, W., Wolkenhauer, O., Cho, K.-H., Yokota, H. (Eds.), *Encyclopedia of Systems Biology*. Springer, New York, NY, http://dx.doi.org/10.1007/978-1-4419-9863-7_1287, 2063–2063.
- Kohl, M., 2011. Standards, databases, and modeling tools in systems biology. In: Hamacher, M., Eisenacher, M., Stephan, C. (Eds.), *Data Mining in Proteomics: From Standards To Applications*. Humana Press, Totowa, NJ, pp. 413–427. http://dx.doi.org/10.1007/978-1-60761-987-1_26.
- Marquardt, W., 2010. *OntoCAPE, First ed.* Springer-Verlag, Berlin Heidelberg.
- Meyer, P., Saez-Rodriguez, J., 2021. Advances in systems biology modeling: 10 years of crowdsourcing dream challenges. *Cell Syst.* 12, 636–653. <http://dx.doi.org/10.1016/j.cels.2021.05.015>.
- Miller, A.K., Britten, R.D., Lloyd, C.M., Nickerson, D.P., Nielsen, P.M., 2013. Cellml. In: Dubitzky, W., Wolkenhauer, O., Cho, K.-H., Yokota, H. (Eds.), *Encyclopedia of Systems Biology*. Springer, New York, NY, pp. 378–381. http://dx.doi.org/10.1007/978-1-4419-9863-7_1093.
- Mostoufi, N., Constantinides, A., 2023. Chapter 8 - linear and nonlinear regression analysis. In: Mostoufi, N., Constantinides, A. (Eds.), *Applied Numerical Methods for Chemical Engineers*. Academic Press, pp. 403–476. <http://dx.doi.org/10.1016/B978-0-12-822961-3.00008-X>.
- Novère, N.L., Hucka, M., Mi, H., Moodie, S., Schreiber, F., Sorokin, A., Demir, E., Wegner, K., Aladjem, M.I., Wimalaratne, S.M., Bergman, F.T., Gauges, R., Ghazal, P., Kawaji, H., Li, L., Matsuoka, Y., Villéger, A., Boyd, S.E., Calzone, L., Courtot, M., Dogrusoz, U., Freeman, T.C., Funahashi, A., Ghosh, S., Jouraku, A., Kim, S., Kolpakov, F., Luna, A., Sahle, S., Schmidt, E., Watterson, S., Wu, G., Goryanin, I., Kell, D.B., Sander, C., Sauro, H., Snoep, J.L., Kohn, K., Kitano, H., 2009. The systems biology graphical notation. *Nat. Biotechnol.* 27 (8), 735–741. <http://dx.doi.org/10.1038/nbt.1558>.
- Oliveira, C.M., Jesus, C.D.F., Ceneviva, L.V.S., Silva, F.H., Cruz, A.J.G., Costa, A.C., 2017. AnaBioPlus: a new package for parameter estimation and simulation of bioprocesses. *Braz. J. Chem. Eng.* 34 (4), 1065–1082. <http://dx.doi.org/10.1590/0104-6632.20170344s20150673>.
- Porubsky, V.L., Goldberg, A.P., Rampadarath, A.K., Nickerson, D.P., Karr, J.R., Sauro, H.M., 2020. Best practices for making reproducible biochemical models. *Cell Syst.* 11 (2), 109–120. <http://dx.doi.org/10.1016/j.cels.2020.06.012>, URL <https://www.sciencedirect.com/science/article/pii/S2405471220302416>.
- Roux-Rouquié, M., Caritey, N., Gaubert, L., Rosenthal-Sabroux, C., 2004. Using the unified modelling language (UML) to guide the systemic description of biological processes and systems. *Biosystems* 75 (1), 3–14. <http://dx.doi.org/10.1016/j.biosystems.2004.03.009>, URL <https://www.sciencedirect.com/science/article/pii/S0303264704000358>, *Computational Systems Biology*.

- Schreiber, F., Le Novère, N., 2013. SBGN. In: Dubitzky, W., Wolkenhauer, O., Cho, K.-H., Yokota, H. (Eds.), *Encyclopedia of Systems Biology*. Springer, New York, NY, pp. 1893–1895. http://dx.doi.org/10.1007/978-1-4419-9863-7_1096.
- Shuler, M., Kargi, F., 2002. *Bioprocess Engineering: Basic Concepts, Second ed.* Prentice Hall PTR, New Jersey.
- Surribas, A., Geissler, D., Gierse, A., Scheper, T., Hitzmann, B., Montesinos, J.L., Valero, F., 2006. State variables monitoring by in situ multi-wavelength fluorescence spectroscopy in heterologous protein production by *Pichia pastoris*. *J. Biotech.* 124 (2), 412–419. <http://dx.doi.org/10.1016/j.jbiotec.2006.01.002>, URL <https://www.sciencedirect.com/science/article/pii/S0168165606000496>.
- Wilkinson, D.J., 2006. *Stochastic Modelling for Systems Biology, First ed.* Chapman and Hall/CRC, New York.
- Yurin, A., Dorodnykh, N., 2020. Personal knowledge base designer: Software for expert systems prototyping. *Softw.* 11, 1–6. <http://dx.doi.org/10.1016/j.softx.2020.100411>.
- Zhang, J., Hunter, A., Zhou, Y., 2013. A logic-reasoning based system to harness bioprocess experimental data and knowledge for design. *Biochem. Eng. J.* 74, 127–135. <http://dx.doi.org/10.1016/j.bej.2013.02.016>.